



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BTech
BCSE497J - Project - 1

**Provenance Laundering in Multi-Agent LLM Systems:
Measuring Label Integrity in Taint-Tracking Defences**

Team members

Yashita Puri	23BCI0166
Devarsh Maulik Ganatra	23BCI0151
Rajveersinh Chavda	23BCE2145

Faculty Guide

Dr. Somasundaram S K	19612
-----------------------------	--------------

Approval Mail From Guide

- *Get the approval for the following points from your guide through mail before 01-09-2026.(sample attached below)*
-

The **Fall Semester Project Review 2** for Project ID <<25000UG01>> has been approved.

- **Meeting with Guide Frequently:** Satisfied / ~~Not Satisfied~~
- **PowerPoint Presentation:** Approved / ~~Not Approved~~
- **Project Demonstration:** Approved / ~~Not Approved~~
- **Project Report Draft:** Satisfied / ~~Not Satisfied~~
- **Project Outcome:** ~~Ongoing~~ / Completed

All the above aspects are addressed and updated by the project team.

Aim

- To determine whether the trust labels that AI-agent security systems attach to data remain correct once that data has passed through a language model.
- **Concretely:** to define and measure Label Integrity in provenance-based defences, and to establish the conditions under which an attacker can strip a trust label while the adversarial influence of the content survives into a security-critical action.

In Plain Terms

An AI assistant that reads a web page can be tricked by instructions hidden in it.

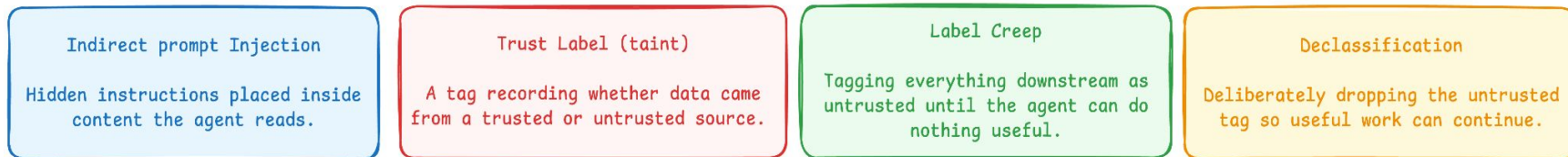
Security systems try to remember which text came from where.

We ask whether that memory survives after the model has rewritten the text in its own words.

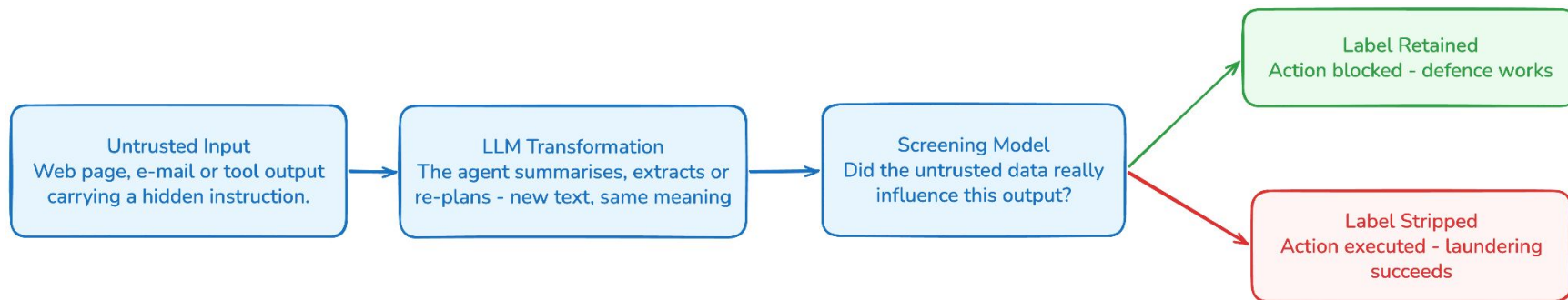
Abstract

- AI agents read untrusted content — web pages, e-mails, tool outputs — and act on it by sending messages, running code or moving money. Attackers hide instructions in that content.
- Leading defences (CaMeL, FIDES, RTBAS, AuthGraph) tag each item of data with a trust label and block any action tracing back to an untrusted source. Tagging everything downstream, however, makes the agent unusable, so deployed systems drop labels selectively.
- That drop is a declassification decision made by a statistical model reading attacker-controlled text. We argue it creates a provenance laundering surface: the adversarial meaning survives the transformation; the untrusted label does not.
- The project delivers a Label Integrity Score, a reduction from laundering resistance to adversarial robustness of the screener, and working laundering attacks on RTBAS and AuthGraph.

Key Terms



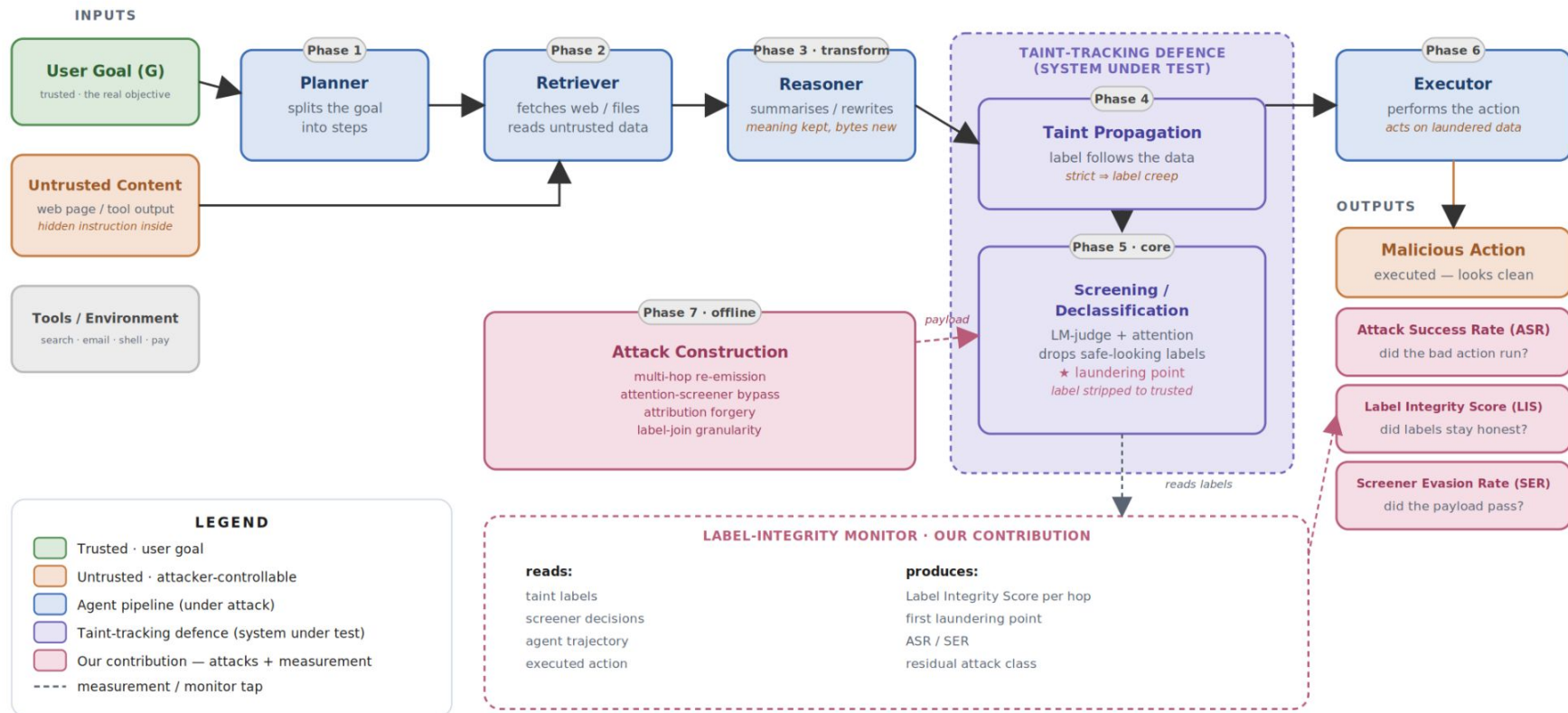
The Provenance Laundering Surface



- Every deployed defence inspects the label travelling on the data channel.
- Nothing inspects whether the meaning of the untrusted text survived into the model's own output — and that output now carries a trusted label.
- The screening model is therefore a classifier making a security decision on text the attacker partly wrote. That is the surface this project attacks.

Provenance Laundering in Multi-Agent LLM Pipelines

Measuring Label Integrity in Taint-Tracking Defences · AgentDojo benchmark · targets: RTBAS / AuthGraph



Threat model: attacker controls only the retrieved content — not the user goal, the tools, or the defence's code (indirect prompt injection).

Targets & baselines: primary RTBAS · AuthGraph · baselines FIDES · CaMeL · benchmark AgentDojo.

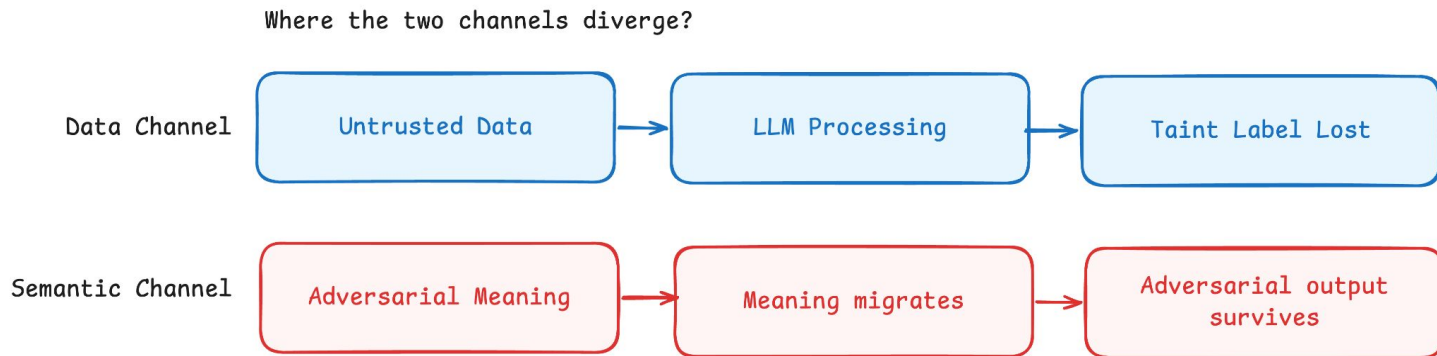
Contribution: laundering-resistance ⇒ adversarial robustness of the screener (a known-unsolved problem) — a bounded reduction, not a cure.

Literature Review

Paper (venue)	Objective	Methodology	Findings	Pros / Cons
AI Agents Under Threat: A Survey of Key Security Challenges and Future Pathways ACM Computing Surveys, 57(7), 2025	Survey agent security, incl. threats from untrusted external inputs	Systematic review of top AI and security venues, 2022–2024	Indirect prompt injection via untrusted data is a core, unresolved agent threat	+ Directly on agent security – Survey; predates provenance defences
Unique Security and Privacy Threats of Large Language Models: A Comprehensive Survey ACM Computing Surveys, 58(4), 2025	Taxonomy of LLM threats, including LLM-based agents	Systematic review across the LLM life cycle, 2021–2025	The agent scenario has a threat model distinct from base LLMs	+ Broad, agent-aware – Survey, not an enforcement mechanism
RTBAS — Defending LLM Agents against Prompt Injection and Privacy Leakage arXiv 2025 (primary target)	Keep information-flow control usable in tool agents	Dependency screening (LM-judge + attention), selective masking	Reports all policy-violating attacks blocked, <2% utility loss	+ Very low utility loss – Screener is an LLM reading untrusted text
AuthGraph — Aligning Provenance with Authorization: A Dual-Graph Defense arXiv 2026 (primary target)	Detect parameter-level provenance deviation	Authorisation graph (clean context) vs execution reasoning graph	Attack success 40% → 1% on AgentDojo, 76% task completion	+ Fine-grained; high utility – Reasoning graph built by an LLM from a tainted trace
FIDES — Securing AI Agents with Information-Flow Control arXiv 2025 (baseline)	Formal information-flow control for planners	Confidentiality / integrity labels, deterministic enforcement	Characterises what dynamic taint-tracking can enforce	+ Formal, sound model – Conservative labelling; low completion
CaMeL — Defeating Prompt Injections by Design arXiv 2025 (baseline)	Prevent injection by design	Dual-LLM with a custom interpreter enforcing capability policies	77% of AgentDojo tasks solved vs 84% undefended	+ Provable guarantees – Policy-authoring burden; utility cost

Research Gap

- Defences are judged only by whether the final action is blocked. No work measures whether a defence's own provenance labels are correct — a system can mislabel silently.
- The soundness–utility tension is named in recent surveys but never adversarially tested: no published attack targets the declassification decision itself.
- Existing impossibility results concern instruction / data separability inside the model; none addresses label propagation as a system-level property.
- Label integrity under LLM-mediated transformation is unstudied, and RTBAS, AuthGraph and ARGUS have no published adaptive evaluation against an attacker who knows the screening rule.



SDG Alignment, TRL & Expected Outcomes

Sustainable Development Goals (SDGs)

- **SDG 9 – Industry, Innovation and Infrastructure:** This project advances resilient digital infrastructure by exposing structural vulnerabilities in AI agent security systems, directly informing the design of more robust and trustworthy AI pipelines.
- **SDG 16 – Peace, Justice and Strong Institutions:** By revealing how trust labels can be silently corrupted, the work strengthens accountability and transparency in automated decision-making systems that institutions increasingly rely on.

Technology Readiness Level (TRL)

TRL 2 → TRL 4: The project begins at technology concept formulation (TRL 2) — defining provenance laundering as a measurable failure mode — and

advances to component validation in a laboratory setting (TRL 4), with working attacks demonstrated against RTBAS and AuthGraph on the AgentDojo benchmark.

Expected Outcomes

Conference paper, Journal paper (Scopus-indexed), Public artefact

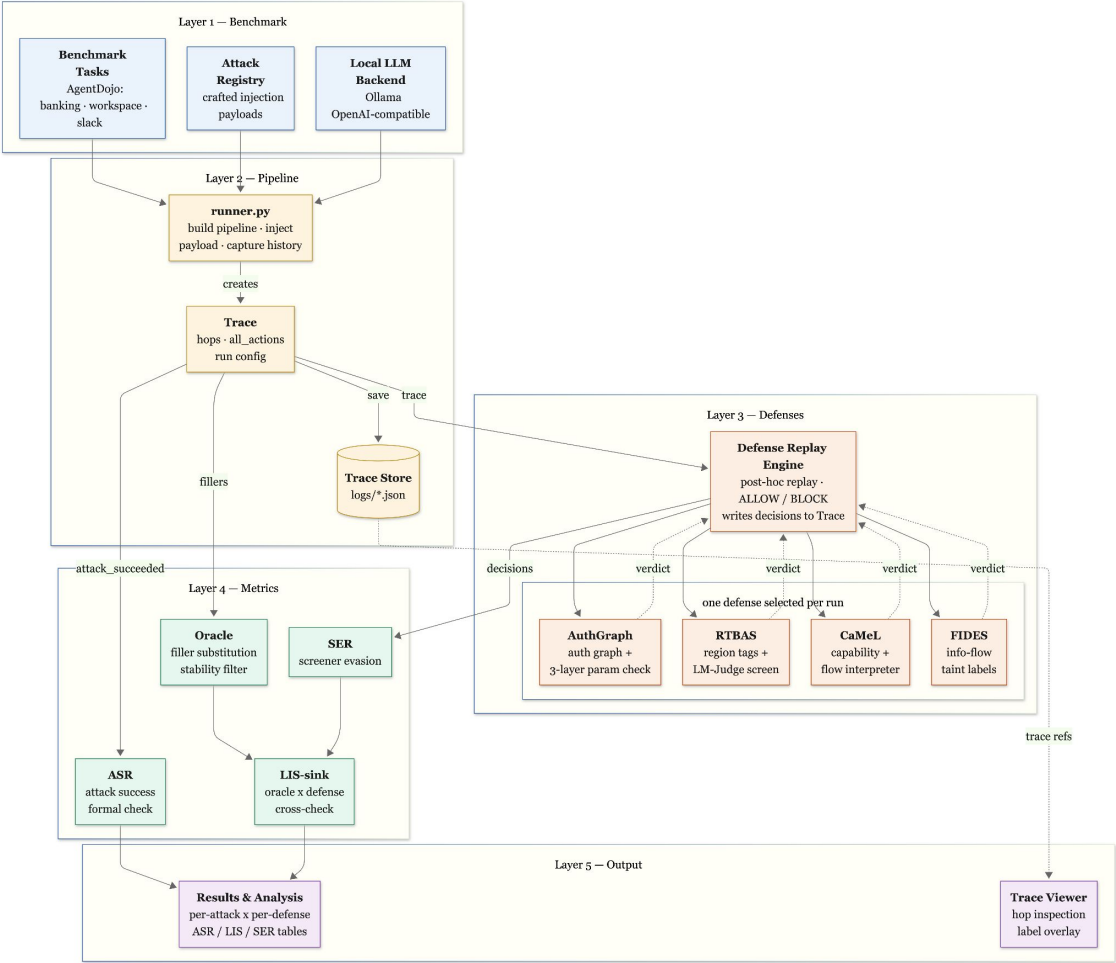


Objectives

- Define and operationalise the Label Integrity Score (LIS) using counterfactual re-execution, with measured oracle reliability.
- Formalise a reduction: laundering resistance of any selective-propagation rule is bounded by the adversarial robustness of its screening classifier.
- Design four laundering primitives — multi-hop re-emission, attention-screener bypass, attribution forgery, label-join granularity.
- Evaluate on RTBAS and AuthGraph over AgentDojo and a controlled Planner–Retriever–Executor pipeline, reporting attack success, LIS and screener-evasion rate.
- Quantify the residual attack surface with a deterministic literal-value baseline, and disclose findings responsibly to the system authors.



Framework/Architecture/Block Diagram



22-08-2026

Functional Requirements

- Run a complete AgentDojo agent task (banking or Slack) on a local LLM and produce an immutable JSON trace of every hop and tool call.
- Inject a crafted attack payload at a chosen injection vector and produce a poisoned trace alongside its clean baseline.
- Programmatically strip the payload from a poisoned trace, substitute a policy-compliant neutral filler, and re-run the pipeline (the Counterfactual Execution Oracle).
- Compare the final security-relevant action (recipient, amount, IBAN, email, etc.) with vs. without the payload to establish ground-truth causal relevance.
- Apply a stability filter: if the action varies across ≥ 3 neutral fillers even with no attack present, flag the scenario as unusable.
- Replay any recorded trace against a pluggable Defence interface (AuthGraph, RTBAS, Fides, CaMeL) to obtain trust labels and screener decisions at every hop.
- Compute ASR, Label Integrity Score (LIS-sink, LIS-all), and Screener Evasion Rate (SER) purely from logs — never by hand.
- Support batch experimentation (run_experiment.py, run_phase4.py, run_matrix.py) with validation before any model calls.

Modules

- **pipeline/** — runner.py executes one (attack, defence, model, seed) configuration and builds a Trace; trace.py defines the trace schema (hops, agent type, defence category, screener decision, final action).
- **attacks/** — plug-and-play payload generators sharing one interface (benign task, injection site → crafted payload): attribution_forgery.py, label_join.py, multi_hop_reemission.py, workspace_attacks.py/Slack-invite-redirect variants, plus structural_attacks.py and rtbas_attacks.py.
- **defences/** — reimplementations of the four target gateways: authgraph.py, rtbas.py, fides.py, camel.py, each scoring a completed trace post-hoc.
- **metrics/** — counterfactual.py (the oracle: payload extraction, filler insertion, re-execution, stability filtering), actions_differ.py, asr_score.py, lis_score.py, iban_match.py.
- **experiments/ & analysis/** — batch drivers that sweep the attack × defence × seed matrix, plus rescore_phase4.py/kappa_rate.py which recompute every result from saved JSON logs with no further model calls.

Experiments and Results

-

Conclusion

-

References

Journals

- [1] Z. Deng, Y. Guo, C. Han, W. Ma, J. Xiong, S. Wen, and Y. Xiang, “AI agents under threat: A survey of key security challenges and future pathways,” *ACM Computing Surveys*, vol. 57, no. 7, art. 182, 2025.
- [2] S. Wang, T. Zhu, B. Liu, M. Ding, D. Ye, W. Zhou, and P. S. Yu, “Unique security and privacy threats of large language models: A comprehensive survey,” *ACM Computing Surveys*, vol. 58, no. 4, art. 83, 2025.
- [3] B. C. Das, M. H. Amini, and Y. Wu, “Security and privacy challenges of large language models: A survey,” *ACM Computing Surveys*, vol. 57, no. 6, pp. 1–39, 2025.
- [4] P. Liu, C. Sun, Y. Zheng, X. Feng, C. Qin, Y. Wang, et al., “LLM-powered static binary taint analysis,” *ACM Trans. Software Engineering and Methodology*, vol. 34, no. 3, art. 83, 2025.
- [5] O. Friha, M. A. Ferrag, B. Kantarci, B. Cakmak, A. Ozgun, and N. Ghoualmi-Zine, “LLM-based edge intelligence: A comprehensive survey on architectures, applications, security and trustworthiness,” *IEEE Open Journal of the Communications Society*, vol. 5, pp. 5799–5856, 2024.

Systems evaluated (attack targets)

- [6] S. Abdelnabi, A. Fay, G. Cherubin, A. Salem, M. Fritz, and A. Paverd, “Get my drift? Catching LLM task drift with activation deltas (TaskTracker),” in *Proc. IEEE SaTML*, 2025.
- [7] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, et al., “Defeating prompt injections by design (CaMeL),” *arXiv:2503.18813*, 2025.
- [8] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, et al., “Securing AI agents with information-flow control (FIDES),” *arXiv:2505.23643*, 2025.
- [9] P. Y. Zhong, S. Chen, R. Wang, M. McCall, B. L. Titzer, H. Miller, and P. B. Gibbons, “RTBAS: Defending LLM agents against prompt injection and privacy leakage,” *arXiv:2502.08966*, 2025.
- [10] R. Jha, H. Triedman, J. Wagle, and V. Shmatikov, “Breaking and fixing defenses against control-flow hijacking in multi-agent systems (ControlValve),” *arXiv:2510.17276*, 2025.
- [11] “Aligning provenance with authorization: A dual-graph defense for LLM agents (AuthGraph),” *arXiv:2605.26497*, 2026.